

AWS All-Region Resource & Cost Audit Guide

[AWS CLI + CloudShell Practical Guide](#)

A beginner-friendly reference for finding what is running in your AWS account, which regions have resources, and what might be costing you money.

Read-only inspection guide — designed so you can look without breaking anything.

Table of Contents

1. Introduction
2. What Went Wrong Originally (Credential Troubleshooting Story)
3. AWS CloudShell
4. AWS Regions
5. Running EC2 Instances Across All Regions
6. All-Region AWS Resource Scanner
7. Running vs Billable — The Most Important Distinction
8. Checking S3 (Global Service)
9. AWS Cost Explorer
10. Checking a Specific Region (Singapore Example)
11. CloudShell Terminal Housekeeping
12. AWS CLI Authentication Troubleshooting
13. Root User Security
14. Read-Only Safety — What NOT to Run
15. Limitations of This Guide
16. Recommended Audit Workflow
17. Quick Reference Cheat Sheet

1. Introduction

This guide teaches you how to inspect your own AWS account safely, using AWS CloudShell and the AWS CLI (Command Line Interface). By the end of it you will be able to find:

- What AWS resources are currently running in your account
- Which AWS regions actually contain resources
- Which services might be generating charges
- How to use AWS CloudShell without needing to set up local credentials
- How AWS CLI authentication works, and how to troubleshoot it
- How to check costs using AWS Cost Explorer
- How to do all of this without accidentally deleting or stopping anything

This guide assumes you already understand basic AWS concepts (what EC2 or S3 roughly is), but are still learning how to use the AWS CLI and AWS CloudShell. Every technical term is explained the first time it appears.

■ Read-only by design

Every command in this guide starts with `describe`, `list`, or `get`. These commands only **read** information — they cannot delete, stop, or change anything in your account.

2. What Went Wrong Originally

This section explains a real troubleshooting situation, so you understand why CloudShell ends up being the easiest starting point for this kind of audit.

2.1 The original attempt

The first attempt was made from Windows PowerShell, running:

```
aws ec2 describe-regions --query "Regions[].RegionName" --output text
```

This returned:

```
AuthFailure
AWS was not able to validate the provided access credentials
```

A second check was run:

```
aws sts get-caller-identity
```

Which returned:

```
InvalidClientTokenId
The security token included in the request is invalid.
```

2.2 What these errors actually mean

AuthFailure

AWS received a request with a signature or access key that it could not validate. In plain English: the credentials being used are wrong, incomplete, or corrupted.

InvalidClientTokenId

The Access Key ID being sent does not match any key AWS recognizes. This usually means the key was deleted, mistyped, or belongs to a different AWS account than expected.

Both errors point to the same underlying problem: the AWS CLI on that machine was using an access key that AWS no longer accepts.

2.3 Checking where the CLI was getting its credentials from

The next step was to check where AWS CLI was actually reading its credentials from:

```
aws configure list
```

This command shows which **profile** is active, and where each credential value (access key, secret key, region) is being loaded from — for example a config file, an environment variable, or nothing at all. In this case, it showed the credentials were coming from the `shared-credentials-file`, under the default profile.

2.4 Key terms explained

Default profile	AWS CLI supports multiple saved sets of credentials, called profiles . If you don't specify one, AWS CLI uses the profile named <code>default</code> .
Shared credentials file	A plain text file on your computer (usually <code>~/.aws/credentials</code> on Mac/Linux or <code>C:\Users\<you>\.aws\credentials</code> on Windows) where AWS CLI stores Access Key IDs and Secret Access Keys you have configured with <code>aws configure</code> .
Environment variables	Values like <code>AWS_ACCESS_KEY_ID</code> that can be set at the operating system level. If present, they override whatever is in the shared credentials file.

Environment variables were checked with:

```
Get-ChildItem Env:AWS*
```

Nothing was returned, which confirmed no environment variables were overriding anything — the problem was specifically the stored key in the shared credentials file being invalid.

2.5 Why the old secret key couldn't just be re-checked

AWS only ever shows you the full Secret Access Key **once**, at the exact moment it is created. After that, the AWS Console and CLI only ever show a masked, partial version of it (for example ending in a few visible characters). This is a deliberate security design — if AWS let you view a full secret key again later, anyone who gained temporary access to your account or screen could steal it permanently.

This means that if a secret key is lost, mistyped, or no longer works, there is no way to "recover" it — the only options are to create a brand new key, or use a different authentication method entirely (which is exactly what led to using CloudShell).

2.6 Why a root-user access key was avoided

The IAM Users page showed no IAM users existed yet, which meant creating an access key would have had to be done on the **root user** — the original, all-powerful account owner identity. AWS explicitly recommends against creating long-term access keys for the root user, because a leaked root key gives an attacker unlimited control over the entire account, including billing.

Instead of creating a root access key just to run some inspection commands, the decision was made to use **AWS CloudShell**, which is covered in the next section.

3. AWS CloudShell

AWS CloudShell is a terminal that runs directly inside your web browser, launched from the AWS Console. It comes with the AWS CLI already installed, and — most importantly — it is automatically authenticated using the identity you are currently logged into the AWS Console with.

3.1 Why this is useful

- No need to create, store, or manage any access keys on your own computer
- No risk of leaking long-term credentials in files or environment variables
- Automatically uses your current Console login session
- Comes pre-installed with AWS CLI, so there is nothing to set up
- Temporary by nature — the credentials CloudShell uses are short-lived session credentials, not permanent keys

3.2 How to open CloudShell

- 1 Log in to the AWS Console in your web browser
- 2 Look for the CloudShell icon in the top navigation bar (it looks like a small terminal/prompt icon, usually next to the notification bell)
- 3 Click it — a terminal panel opens at the bottom of the browser window
- 4 Wait a few seconds while it provisions a small temporary environment for you

3.3 Confirming your identity

Once CloudShell opens, run:

```
aws sts get-caller-identity
```

This command asks AWS "who am I, according to the credentials currently being used?" It returns three important values:

UserId	A unique identifier for the specific IAM identity or root user making the request.
Account	The 12-digit AWS account number. This is the most important field to check — make sure it matches the account you actually intend to inspect.
Arn	The full "Amazon Resource Name" of the identity — a structured string identifying exactly who or what is making the request (root user, IAM user, or IAM role).

■ Always verify the account number

If you manage more than one AWS account, always check the Account field before running any resource scan. Running commands against the wrong account wastes time and can be confusing when resources don't show up where you expect.

4. AWS Regions

An AWS **Region** is a separate physical location around the world where AWS runs data centers. Each region operates largely independently — a resource created in one region generally does not exist in, or affect, any other region.

ap-southeast-1	Singapore
us-east-1	N. Virginia, USA
eu-west-1	Ireland

This regional separation is exactly why resources can quietly exist somewhere you forgot about — for example, you might remember creating a server in Singapore, but forget that you also spun one up in N. Virginia while following an old tutorial. Each region needs to be checked separately, which is why an **all-region scan** is so useful.

4.1 Listing all available regions

```
aws ec2 describe-regions --query 'Regions[].RegionName' --output text
```

What this command does:

aws ec2 describe-regions	Asks AWS for the full list of regions available to this account.
--query 'Regions[].RegionName'	Filters the response down to just the region name field, instead of printing a large block of JSON.
--output text	Formats the result as plain space-separated text instead of JSON, which is easier to read and to use inside a loop.

5. Running EC2 Instances Across All Regions

This command checks every region, one at a time, for any EC2 instance that is currently in the running state:

```
for region in $(aws ec2 describe-regions --query 'Regions[].RegionName' --output text); do
  instances=$(aws ec2 describe-instances \
    --region "$region" \
    --filters Name=instance-state-name,Values=running \
    --query 'Reservations[].Instances[].InstanceId' \
    --output text)

  if [ -n "$instances" ]; then
    echo "==== $region ====="
    echo "$instances"
  fi
done
```

5.1 Line-by-line explanation

for region in \$(...); do	Starts a loop that runs once for every region name returned by the describe-regions command from the previous section.
aws ec2 describe-instances --region "\$region"	Asks AWS to list EC2 instances, but only inside the current region being looped through.
--filters Name=instance -state-name,Values=running	Tells AWS to only return instances that are currently switched on and running — stopped instances are excluded here.
--query '...InstanceId'	Extracts just the Instance ID (a short unique identifier) from the response, instead of the full details.
if [-n "\$instances"]; then ... fi	Only prints something for this region if at least one running instance was actually found — regions with nothing running stay silent.

5.2 What the different situations look like

- **Region with a running instance:** prints the region name in a header line, followed by one or more Instance IDs.
- **Region with no running instance:** nothing is printed at all for that region — this is normal and expected for most regions.
- **Multiple instances in one region:** all matching Instance IDs are listed under the same region header.
- **Permission error:** if the current identity does not have permission to check a particular region or service, AWS CLI prints an AccessDenied or UnauthorizedOperation error for that specific check instead of silently skipping it.

Example output:

```
==== ap-southeast-1 ====
i-0a1b2c3d4e5f67890

==== us-east-1 ====
i-0f9e8d7c6b5a43210
i-01122334455667788
```

6. All-Region AWS Resource Scanner

This expanded script checks several common AWS services across every region in a single run. It is the core tool in this guide.

```
for r in $(aws ec2 describe-regions --query 'Regions[].RegionName' --output text); do
echo ""
echo "===== $r ====="

echo "--- EC2 ---"
aws ec2 describe-instances --region $r \
--filters Name=instance-state-name,Values=running \
--query
'Reservations[].Instances[].{ID:InstanceId,Type:InstanceType,State:State.Name}' \
--output table 2>/dev/null

echo "--- RDS ---"
aws rds describe-db-instances --region $r \
--query 'DBInstances[?DBInstanceStatus==`available`].{ID:DBInstanceIdentifier,Class:DB
InstanceClass,Status:DBInstanceStatus}' \
--output table 2>/dev/null

echo "--- NAT Gateway ---"
aws ec2 describe-nat-gateways --region $r \
--filter Name=state,Values=available \
--query 'NatGateways[].{ID:NatGatewayId,State:State}' \
--output table 2>/dev/null

echo "--- Load Balancers ---"
aws elbv2 describe-load-balancers --region $r \
--query 'LoadBalancers[?State.Code==`active`].{Name:LoadBalancerName,Type:Type,State:S
tate.Code}' \
--output table 2>/dev/null

echo "--- ECS Services ---"
for c in $(aws ecs list-clusters --region $r --query 'clusterArns[]' --output text
2>/dev/null); do
aws ecs list-services --region $r --cluster "$c" \
--query 'serviceArns[]' --output text 2>/dev/null
done

echo "--- Lambda ---"
aws lambda list-functions --region $r \
--query 'Functions[].{Name:FunctionName,Runtime:Runtime}' \
--output table 2>/dev/null

echo "--- ElastiCache ---"
aws elasticache describe-cache-clusters --region $r \
--query 'CacheClusters[].{ID:CacheClusterId,Status:CacheClusterStatus,Engine:Engine}' \
\
--output table 2>/dev/null

echo "--- OpenSearch ---"
aws opensearch list-domain-names --region $r \
--query 'DomainNames[].{Name:DomainName}' \
--output table 2>/dev/null

echo "--- EBS Volumes ---"
```



```

aws ec2 describe-volumes --region $r \
--query 'Volumes[].{ID:VolumeId,Size:Size,State:State}' \
--output table 2>/dev/null

echo "--- Elastic IPs ---"
aws ec2 describe-addresses --region $r \
--query 'Addresses[].{IP:PublicIp,Instance:InstanceId}' \
--output table 2>/dev/null
done

echo ""
echo "===== S3 (global) ====="
aws s3 ls 2>/dev/null

```

The `2>/dev/null` at the end of many lines simply hides error messages (for example, permission errors for a service you've never used) so the output stays clean and readable.

6.1 What each section checks

EC2 — Elastic Compute Cloud

What it is	Virtual servers.
What the command checks	Lists running instances with their ID, instance type (size), and state.
Why it might cost money	Charged per hour/second while running, based on instance type.
Scope	Regional

RDS — Relational Database Service

What it is	Managed databases (MySQL, PostgreSQL, etc).
What the command checks	Lists databases currently in the available status.
Why it might cost money	Charged continuously while the database exists and is running, based on instance class and storage.
Scope	Regional

NAT Gateway

What it is	A managed gateway that lets private resources reach the internet.
What the command checks	Lists NAT Gateways currently in the available state.

Why it might cost money	Charged an hourly fee plus a per-GB data processing fee, for as long as it exists — even with very little traffic.
Scope	Regional

Load Balancers

What it is	Distributes incoming traffic across multiple servers.
What the command checks	Lists load balancers currently active.
Why it might cost money	Charged hourly while active, plus usage-based fees.
Scope	Regional

ECS — Elastic Container Service

What it is	Runs containerized applications.
What the command checks	Lists clusters, then lists any services running inside each cluster.
Why it might cost money	The underlying compute (EC2 or Fargate) used by ECS is what generates charges.
Scope	Regional

Lambda

What it is	Runs small pieces of code without managing a server.
What the command checks	Lists all Lambda functions and their runtime (e.g. Python, Node.js).
Why it might cost money	Usually charged per invocation and execution time — typically very cheap, but worth knowing what exists.
Scope	Regional

ElastiCache

What it is	Managed in-memory caching (Redis/Memcached).
What the command checks	Lists cache clusters, their status, and engine.

Why it might cost money	Charged continuously per hour while the cluster exists, similar to RDS.
Scope	Regional

OpenSearch

What it is	Managed search and analytics engine.
What the command checks	Lists OpenSearch domain names.
Why it might cost money	Charged continuously while the domain exists, based on instance type and storage.
Scope	Regional

EBS — Elastic Block Store

What it is	Virtual hard drives attached to EC2 instances.
What the command checks	Lists volumes with their size and state.
Why it might cost money	Charged per GB of storage per month, even if the volume is not attached to a running instance.
Scope	Regional

Elastic IP

What it is	A fixed public IPv4 address you can attach to resources.
What the command checks	Lists allocated public IPs and which instance (if any) they are attached to.
Why it might cost money	Can be charged when not efficiently attached/used, depending on current AWS pricing rules.
Scope	Regional

S3 — Simple Storage Service

What it is	Object storage for files.
What the command checks	Lists all S3 buckets in the account.

Why it might cost money	Charged for storage used, requests made, and data transferred out.
Scope	Global — not tied to a single region in the same way as the others

7. Running vs Billable — The Most Important Distinction

■ Key idea

A resource does **not** always have to be "running" to generate AWS charges. Several services keep billing you even while switched off, stopped, or sitting idle.

EC2

A **stopped** EC2 instance stops charging for compute time, but its attached EBS volumes keep existing and keep being charged for storage, unless they are deleted.

Elastic IP

Public IPv4 addresses can incur charges depending on how they are allocated and used — including in some cases even when attached, and especially when **not** attached to a running resource. Always check whether an Elastic IP is actually in use.

NAT Gateway

NAT Gateways charge an hourly fee simply for existing, plus a separate per-GB fee for any data that passes through them — regardless of how "busy" they are.

Load Balancers

An active load balancer continues generating hourly charges even if it currently has very little or no traffic passing through it.

S3

S3 has three separate cost dimensions: the amount of data stored, the number of requests made (uploads, downloads, listings), and the data transferred out to the internet. A bucket can quietly accumulate storage charges over time even if nobody is actively using it.

■ Practical takeaway

When auditing your account, don't just look for things that are "running." Also check stopped EC2 instances (for leftover EBS volumes), unattached Elastic IPs, existing NAT Gateways, and S3 buckets — all of these can bill you while looking harmless.

8. Checking S3 (Global Service)

S3 is checked separately from the rest of the scanner because it is a **global** service — buckets are listed under your account as a whole, not per-region in the same way EC2 instances are (even though each bucket does have an underlying region where its data physically lives).

```
aws s3 ls
```

This lists every bucket in your account along with its creation date. It does not show storage size or cost — for that, Cost Explorer or the S3 Console's storage metrics are needed.

9. AWS Cost Explorer

Cost Explorer is AWS's built-in tool for analyzing what you have actually been billed for, broken down by service, time period, and more. It can be accessed through the CLI as well as the Console.

```
aws ce get-cost-and-usage \
--time-period Start=$(date -u -d '30 days ago' +%Y-%m-%d),End=$(date -u +%Y-%m-%d) \
--granularity MONTHLY \
--metrics UnblendedCost \
--group-by Type=SERVICE
```

Start / End	Defines the date range to analyze. Here, <code>Start</code> is calculated as 30 days before today, and <code>End</code> is today's date, so this covers the last month.
MONTHLY	The granularity — how the results are grouped over time. Monthly gives one total per calendar month in the range, rather than daily figures.
UnblendedCost	A standard AWS cost metric representing the actual cost of usage as it was incurred, without any averaging or discount-sharing adjustments across accounts. For a single personal account, this is generally the simplest cost figure to understand.
Group-by Type=SERVICE	Breaks the total cost down into separate lines per AWS service (EC2, S3, RDS, etc.), so you can see which service is responsible for how much of the bill.

If this command returns a permissions error, the identity you are using does not have Cost Explorer permissions. In that case, use the Console alternative instead:

■ Console alternative

AWS Console → Billing and Cost Management → Cost Explorer

10. Checking a Specific Region (Singapore Example)

Sometimes you already know which region to check, and don't need the full all-region loop. Here is an example checking only Singapore:

```
aws ec2 describe-instances \
--region ap-southeast-1 \
--filters Name=instance-state-name,Values=running \
--query
'Reservations[].Instances[].{ID:InstanceId,Type:InstanceType,IP:PublicIpAddress}' \
--output table
```

`ap-southeast-1` is the AWS region code for Singapore. This command lists any currently running EC2 instances there, along with their instance type and public IP address.

11. CloudShell Terminal Housekeeping

Clear the visible terminal screen:

```
clear
```

or the keyboard shortcut:

```
Ctrl + L
```

Either of these only clears what is currently displayed on screen. It does **not** delete or affect the commands you have already run — it's purely visual.

Clear shell history

```
history -c  
history -w
```

The **terminal screen** and the **shell history** (the saved list of commands you've typed, which you can scroll through with the up-arrow key) are two separate things. `history -c` clears the in-memory history list, and `history -w` writes that cleared state to the history file so it doesn't come back.

■ Be careful with destructive commands

Never run a command like `rm -rf ~` unless you fully understand exactly what it does. That specific command permanently deletes your entire home directory. This guide never asks you to run anything like that.

12. AWS CLI Authentication Troubleshooting

Three commands are useful whenever AWS CLI authentication misbehaves:

```
aws configure list
```

Shows which profile is active and where each credential value is being sourced from (config file, environment variable, or not set at all).

```
aws sts get-caller-identity
```

Confirms which identity (and which AWS account) the current credentials actually belong to — the quickest way to check whether authentication is working at all.

```
Get-ChildItem Env:AWS*
```

A Windows PowerShell command that lists any environment variables starting with AWS, useful for checking whether something is silently overriding your normal credentials.

12.1 Key terms

Access Key ID	A public identifier for a set of credentials — similar in spirit to a username. Not secret on its own, but only useful paired with its matching secret key.
Secret Access Key	The actual secret half of a credential pair, used to cryptographically sign requests. Shown in full only once, at creation time.
Session Token	An additional, short-lived value used alongside temporary credentials (such as those CloudShell or an IAM role provides). Without the matching session token, temporary access keys alone will not work.
AWS CLI profile	A named set of saved credentials/settings. The default profile is used automatically unless you specify a different one with <code>--profile</code> .
Environment variables	Values set at the OS/session level (like <code>AWS_ACCESS_KEY_ID</code>) that, if present, override whatever is stored in the shared credentials file.
Shared credentials file	The local file where <code>aws configure</code> saves your keys, organized by profile.
AWS Console session	Your logged-in browser session in the AWS Console. CloudShell borrows this session automatically, which is why it doesn't need separate keys.

One important nuance: because environment variables override the shared credentials file, a stray environment variable left over from an old project can cause confusing authentication failures even if your saved profile is completely correct — always check both.

Temporary credentials (like the ones CloudShell and IAM roles use) are designed to **expire** automatically after a set period, usually a matter of hours. This is a security feature, not a bug — if a command that worked earlier suddenly starts failing with an authentication error, an expired session is one of the most common causes.

13. Root User Security

The **root user** is the original identity created when an AWS account is first set up, tied directly to the email address used to create the account. It has unrestricted permissions over everything in the account, including billing, and — critically — it cannot be permission-restricted the way IAM users and roles can.

- **Why root has extremely powerful permissions:** by design, there is no way to limit what the root user can do — it can access, change, or delete anything.
- **Why long-term root access keys are discouraged:** a leaked root access key gives permanent, unrestricted control of the entire account to whoever has it, with no way to scope it down afterward except deleting the key entirely.
- **Why IAM users/roles are preferred instead:** IAM identities can be given only the specific permissions they actually need, and can be individually disabled or deleted without affecting the root account.
- **Why CloudShell is convenient here:** it lets you run inspection commands using your current Console session's identity, without ever needing to create a long-term access key at all — root or otherwise.

■ Never publish AWS credentials to GitHub

Accidentally committing credentials to a public (or even private) GitHub repository is one of the most common ways AWS accounts get compromised and run up large unexpected bills. Never commit any of the following:

```
AWS_ACCESS_KEY_ID
AWS_SECRET_ACCESS_KEY
AWS_SESSION_TOKEN
.env files containing credentials
private keys
passwords
```

If you ever do accidentally commit a credential, treat it as immediately compromised: delete/deactivate that access key in the IAM console right away, even if you also remove it from the repository — removing it from git history alone is not enough, since it may already have been scraped by automated bots.

14. Read-Only Safety — What NOT to Run

Every command used for the audit in this guide is intentionally read-only. AWS CLI commands generally fall into recognizable patterns based on their prefix:

describe-*	Reads and returns details about existing resources. Safe — does not modify anything.
list-*	Lists existing resources (names, IDs). Safe — does not modify anything.
get-*	Retrieves a specific piece of information or data. Safe — does not modify anything.

By contrast, these are examples of destructive commands that this guide never asks you to run:

■ **Do not run these unless you specifically intend to delete something**

`terminate-instance` — permanently deletes an EC2 instance

`delete-db-instance` — permanently deletes an RDS database

`delete-nat-gateway` — deletes a NAT Gateway

`release-address` — releases an Elastic IP

`delete-bucket` — deletes an S3 bucket

The audit script in this guide should never delete, stop, or modify any resource. It only **observes** your account. Any decision to stop or delete something should be made manually, afterward, and separately.

15. Limitations of This Guide

■ **Important**

AWS offers a very large number of services, and there is no single, universal AWS CLI command that perfectly lists every possible resource across every service in one shot.

The scanner in this guide covers the services most commonly responsible for running resources and unexpected costs for a typical personal or small-project account — but it is **not a guaranteed complete inventory** of everything that could exist or generate charges in an AWS account.

For a more complete picture, combine this scanner with:

- 1 All-region resource scanning (this guide's core script)
- 2 Direct S3 inspection
- 3 AWS Cost Explorer, to catch anything billed that wasn't directly scanned
- 4 AWS Billing dashboard, for the overall account-level total
- 5 AWS Budgets and billing alerts, to get notified proactively instead of checking manually
- 6 Service-specific checks for anything you know you've used but isn't covered above (for example CloudFront, Route 53, or SageMaker)

16. Recommended Audit Workflow

A simple order to follow each time you want to audit your account:

1	Open AWS CloudShell
2	Run: <code>aws sts get-caller-identity</code>
3	Verify the AWS account number is the one you meant to check
4	List all regions
5	Run the all-region resource scanner
6	Check S3 separately
7	Check Cost Explorer (or the Billing console)

8	Investigate anything unexpected you find
9	Only then consider stopping or deleting anything — deliberately, and one resource at a time

17. Quick Reference Cheat Sheet

The most important commands from this guide, in one place.

Identity & setup

```
aws sts get-caller-identity
aws configure list
```

Regions

```
aws ec2 describe-regions --query 'Regions[].RegionName' --output text
```

Running EC2 instances in one region

```
aws ec2 describe-instances \
--region ap-southeast-1 \
--filters Name=instance-state-name,Values=running \
--query
'Reservations[].Instances[].{ID:InstanceId,Type:InstanceType,IP:PublicIpAddress}' \
--output table
```

Running EC2 instances across all regions

```
for region in $(aws ec2 describe-regions --query 'Regions[].RegionName' --output
text); do
instances=$(aws ec2 describe-instances --region "$region" \
--filters Name=instance-state-name,Values=running \
--query 'Reservations[].Instances[].InstanceId' --output text)
if [ -n "$instances" ]; then
echo "==== $region ====="
echo "$instances"
fi
done
```

S3 buckets (global)

```
aws s3 ls
```

Cost Explorer — last 30 days by service

```
aws ce get-cost-and-usage \
--time-period Start=$(date -u -d '30 days ago' +%Y-%m-%d),End=$(date -u +%Y-%m-%d) \
--granularity MONTHLY \
--metrics UnblendedCost \
--group-by Type=SERVICE
```

Terminal housekeeping

```
clear
history -c
history -w
```

Windows: check AWS environment variables

```
Get-ChildItem Env:AWS*
```

■ Remember

This guide is for inspection only. Never run terminate/delete/release commands unless you have specifically decided, on purpose, to remove that exact resource. Never commit AWS credentials to GitHub or any other repository.